Find and Apply Data Diff (Delta Ingestion with Lookups)

Context

In real-world systems, data is rarely loaded into empty tables.

Instead:

- A destination table already contains **existing records**
- New data arrives periodically (files, APIs, streams)
- Some records are **new**
- Some already exist
- Some may be **partially updated**

Blindly re-inserting all records causes:

- Duplicates
- Referential integrity issues
- Performance problems

Your task is to design and implement a **backend service** that:

1. **Computes the delta** between incoming data and existing data
2. **Applies only the required inserts**
3. Correctly handles **lookup / reference tables**

Problem Statement

You are given an existing relational database with a `customers` table.

Periodically, a new data file arrives containing:

- Customers that already exist in the table
- Customers that do not yet exist and must be inserted

Your goal is to:

- Identify **only the delta (new customers)**
- Insert **only those records**
- Ensure all **lookup dependencies** are correctly resolved

Database Schema

Customers (Destination Table)

```
customers (
  customer_id      BIGINT PRIMARY KEY,
  external_id      TEXT UNIQUE,
  name             TEXT,
  email            TEXT,
  country_id       BIGINT REFERENCES countries(id),
  status_id        BIGINT REFERENCES customer_status(id),
```

```
  created_at       TIMESTAMP
)

```

Lookup Tables

```
countries (
  id    BIGINT PRIMARY KEY,
  code  TEXT UNIQUE,  -- e.g. "US", "IN"
  name  TEXT
)

customer_status (
  id    BIGINT PRIMARY KEY,
  code  TEXT UNIQUE,  -- e.g. "ACTIVE", "INACTIVE"
  name  TEXT
)
```

Incoming Data (File / API Payload)

Incoming data is provided as JSON:

```
[
  {
    "external_id": "cust_001",
    "name": "Alice",
    "email": "alice@example.com",
    "country_code": "US",
    "status_code": "ACTIVE"
  },
  {
    "external_id": "cust_999",
    "name": "Bob",
    "email": "bob@example.com",
    "country_code": "IN",
    "status_code": "ACTIVE"
  }
]

```

Some of these customers already exist in `customers`.

Some do not.

Your Task

Core Requirements

1. **Delta Detection**
   - Compare incoming data with existing records
   - Identify customers that **do not exist** in the destination table
   - Existing customers must NOT be re-inserted
2. **Insert Only Delta**
   - Insert only new customers
   - Maintain referential integrity with lookup tables
3. **Lookup Resolution**
   - Resolve `country_code → countries.id`
   - Resolve `status_code → customer_status.id`

- Handle missing or invalid lookup values gracefully

4. **Idempotency**
   - Running the same input twice must not create duplicates

API Contract

You may expose an API like:

```
POST /customers/ingest

```

Response:

```
{
  "received": 1000,
  "inserted": 120,
  "skipped_existing": 880,
  "failed": 0
}

```

Constraints

- Incoming batch size: up to **100k records** (you have to implement chunking on your end - the original source data can have up to 10 million records)
- Customers table: up to **10 million records**
- Solution must be efficient and scalable

Non-Functional Requirements

- Avoid N+1 queries
- Use bulk operations where appropriate
- Clean transaction handling
- Clear error reporting
- Well-structured code

What We Are Evaluating

Data Modeling & Query Design

- How existence is checked (keys, indexes)
- Efficient diff computation
- Bulk insert strategy

Lookup Handling

- Caching vs querying
- Failure handling for missing lookups
- Ordering of operations

Correctness

- Idempotency

- Referential integrity
- Partial failure handling

Edge Cases to Consider (Not Exhaustive)

- Duplicate records in incoming data
- Missing lookup values
- Case sensitivity in codes
- Concurrent ingestion jobs
- Partial batch failures

Bonus (Optional – Strong Signal)

1. **Upsert vs Insert-only Design**
   - Explain why you chose one
2. **Dry-Run Mode**
   - Return what *would* be inserted without writing
3. **Metrics**
   - Time taken
   - Rows scanned
   - Cache hit ratio
4. **Rollback Strategy**
   - If part of the batch fails